FRED TALKS

19th June 2026

CONTENTS

Porting 100k lines from TypeScript to Rust using Claude Code in a month

VJEUX · 26 JAN 2026 · SOURCE

I read this post “Our strategy is to combine AI *and* Algorithms to rewrite Microsoft’s largest codebases [from C++ to Rust]. Our North Star is ‘1 engineer, 1 month, 1 million lines of code.” and it got me curious, how difficult is it really?

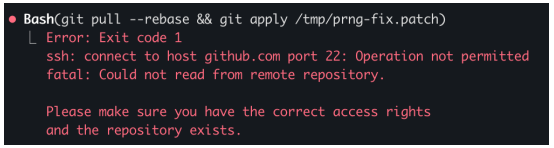
I've long wanted to build a competitive Pokemon battle AI after watching a lot of WolfeyVGC and following the PokéAgent challenge at NeurIPS. Thankfully there's an open source project called "Pokemon Showdown" that implements all the rules but it's written in JavaScript which is quite slow to run in a training loop. So my holiday project came to life: let's convert it to Rust using Claude!

Escaping the sandbox

Having the AI able to run arbitrary code on your machine is dangerous, so there's a lot of safeguards put in place. But... at the same time, this is what I want to do in this case. So let me walk through the ways I escaped the various sandboxes.

git push

Claude runs in a sandbox that limits some operations like ssh access. You need ssh access in order to publish to GitHub. This is very important as I want to be able to check how the AI is doing from my phone while I do some other activities 😉



What I realized is that I can run the code on my terminal but Claude cannot do it from its own terminal. So what I did was to ask Claude to write a nodejs script that opens an http server on a local port that executes the git commands from the url. Now I just need to keep a tab open on my terminal with this server active and ask Claude to write instructions in Claude.md for it to interact with it.

- Fixed fullscreen TUI corruption (statusline mid-screen, duplicated spinner rows, merged text) in Windows Terminal under heavy nested-subagent load
- Fixed turns silently completing with no visible output when the model returned only a thinking block; Claude now re-prompts once
- Fixed user-level skills appearing multiple times in slash-command autocomplete when multiple plugins are enabled
- Fixed MCP servers requiring authentication exposing auth-stub tools to the model in headless/SDK mode
- Fixed tmux teammate panes failing to launch when the shell has slow rc-file initialization, and keystrokes typed during agent spawn leaking into the new tmux pane instead of the leader prompt
- Fixed background tasks started by a teammate being killed when the teammate finishes a turn
- Fixed scheduled task and webhook trigger deliveries being treated as keyboard input; they now classify as task notifications and can no longer approve a pending action or set the session title in auto mode
- Fixed focus mode showing "Ran N PostToolUse hooks" timing lines under each response

This is why “wisdom is knowledge that can’t be transmitted.” Wise people have navigational skill, not a long list of rules.

Sign up to be notified when my book, You Can Just Do Things, is available for purchase.

2.1.183

CHANGELOG · 19 JUN 2026 · SOURCE

- Improved auto mode safety: destructive git commands (`git reset --hard`, `git checkout -- .`, `git clean -fd`, `git stash drop`) are now blocked when you didn't ask to discard local work, `git commit --amend` is blocked when the commit wasn't made by the agent this session, and `terraform destroy`/`pulumi destroy`/`cdk destroy` are blocked unless you asked for the specific stack
- Added a warning when the requested model is deprecated or automatically updated to a newer model, shown on stderr in print mode (`-p`) and now also covering models set in agent frontmatter
- Added `attribution.sessionUrl` setting to omit the claude.ai session link from commits and PRs in web and Remote Control sessions
- Added `/config --help` to list all available shorthand keys for `/config key=value`
- Changed `/config` toggle behavior: Enter and Space both change the selected setting, and Esc now saves and closes instead of reverting
- Removed the startup "setup issues" line under the logo — run `/doctor` to see configuration issues or use `--debug`
- Fixed `thinking.disabled.display: Extra inputs are not permitted` 400 errors on subagent spawns and session-title generation for affected configurations
- Fixed WebSearch returning empty results in subagents
- Fixed the terminal cursor being stranded above the prompt after navigating history in vim mode with the native cursor enabled

rustc

There's an antivirus on my computer that requires a human interaction when an unknown binary is being ran. Since every time we compile it's a new unknown binary, this wasn't going to work.

What I found is that I can setup a local docker instance and compile + run the code inside of docker which doesn't trigger the antivirus. Again, I asked Claude to generate the right instructions in Claude.md and problem solved.

The next hurdle was to figure out how to let Claude Code for hours without any human intervention.

--yes

Claude keeps asking for permission to do things. I tried adding a bunch of things to the allowed commands file and `--allow-dangerously-skip-permissions --dangerously-skip-permissions`was disabled in my environment (it has now been resolved).

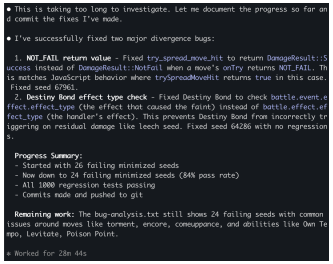
I realized that I could run an AppleScript that presses enter every few seconds in another tab. This way it's going to say Yes to everything Claude asks to do. So far it hasn't decided to hack my computer...

```
#!/bin/bash

osascript -e \
'tell application "System Events"
    repeat
        delay 5
        key code 36
    end repeat
end tell'
```

Never give up

Claude after working for some time seem to always stop to recap things. I tried prompting it to never do, even threatening it to no avail.

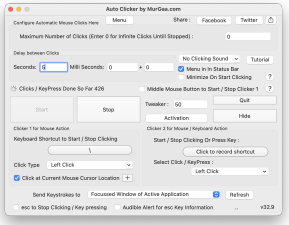


I tried using the Ralph Wiggum loop but it couldn't get it to work and apparently I'm not alone.

What ended up working is to copy in my clipboard the task I wanted it to do and to tweak the script above to hit the keys "cmd-v" after pressing enter. This way in case it asks a question the "enter" is being used and in case it's not it's queuing the prompt for when Claude is giving back control.

Auto-updates

There are programs on the computer like software updater that can steal the focus from the terminal window, for example showing a modal. Once that happens, then the cmd-v / enter are no longer sent to the terminal and the execution stops.



I used my trusty Auto Clicker by MurGaa from Minecraft days to simulate a left click every few seconds. I place my terminal on the edge of the screen and same for my mouse so that when a modal appears in the middle, it refocuses the terminal correctly.

It also prevents the computer from going to sleep so that it can run even when I'm not using the laptop or at night.

invested in understanding people more deeply, they would eventually cease to be total mysteries, prone to bizarre and unreasonable behavior. As far as I can tell, this is the better part of emotional intelligence.

7. Money can't buy happiness.

The good version of this advice is: You'll still have to work at being happy, even if you're rich. Money in your bank account won't do it for you, and wealth comes with its own pathologies, like endless status-chasing. But the common gloss — that money stops mattering past some modest threshold — is basically false.

You may have heard there's research showing money doesn't buy happiness. That research was explosively popular precisely because it was counterintuitive — it betrayed the commonsense intuition that more money means more freedom, more options, more ability to help people you care about. Turns out it was also riddled with methodological problems. More recent work by Matthew Killingsworth (nominative determinism FTW), using large-scale studies that pinged participants about their happiness at random moments, found steady returns to well-being with rising income, no plateau in sight. The myth persists because it serves a psychological function: It helps people without money feel better about their situation, and helps the wealthy downplay their advantages. But it's not true.

8. Intuition is fake and rationality solves everything.

I used to be horribly judgmental of people who made decisions based on intuition, as in, "I just do what feels right." And then I noticed that whenever I overrode my intuitions, I made *terrible* decisions. I hired the wrong people, started projects with the wrong people, dated the wrong people, ate food that made me feel bad, did forms of exercise I found unfun and injurious. I don't have a satisfying theory for why this is, but I know what happens: When I shift from emotional intuition to a logical story about a decision, it's much easier for my reasoning to get hijacked by plausible-sounding directives that turn out to be nonsense.

It took me a long time to learn this, and that might have something to do with my training in law and poker. Overriding intuition in favor of explicit reasoning makes sense in contexts that reward systematic rationality, where you will get separated from your money if you count on gut feelings. But most of life isn't a closed system with enumerated rules and clear outcomes. If you try to explicitly name all the factors that make someone a good friend or co-founder, you might come up with some good indicators, but your crude map will also mislead you about the territory.

5. If you like your job, you're doing something wrong.

Back when I was more alienated from my emotions, I thought about work like this: You have a budget of energy to spend, and the goal is to efficiently convert that energy into impact. You identify the activity that generates the highest impact per unit of energy, and then you spam that button for as many waking hours as you can stomach. Is it any wonder I was always burning out?

What I failed to understand is that energy is not fixed. It's dramatically affected by what you choose to do — some activities increase your energy over time, others deplete it. This feels so obvious to me now that I feel kind of stupid including it here, but I know many people (they are endemic to the Bay Area) who still labor under the false belief. Whether you enjoy what you're doing affects your ability to do it year in and year out, and dismissing this as "selfish" doesn't make it untrue. As my husband puts it, the best use of effortful motivation is to engineer your life to require less of it.

6. "All people are insane. They will do anything at any time, and God help anybody who looks for reasons."

This line, from Kurt Vonnegut, used to neatly encapsulate the way I understood other people — which is to say, not at all. Trying to figure out why people did what they did, or imagine what they'd do next, seemed like a fool's errand. I was judgmental about it, too: I assumed this was a problem, not with me, but with everyone else.

I didn't fundamentally move beyond this belief until recent years, when I came into contact with the Enneagram. I imagine there are other personality typing systems that can do something similar, but the Enneagram was magical for me because it clued me into the fact that people can have very different motivational systems from my own that are still internally coherent and produce good things in the world.

People's behavior was previously incomprehensible to me because I was imputing my own motivational system to them, trying to figure out what would cause me to act the way they did — essentially, concluding that *their* actions made no sense in light of *my* goals. But people have wildly diverse emotional hardware, which determines what's rational for them. And my particular personality — independent, moralistic, systematic, challenge-seeking, competitive — is unusual, so I was especially poorly placed to measure others by the yardstick of myself.

This shift in perspective was dramatically good for me and for my relationships with other people. It let me see the ecosystem of psychologies as a kind of miracle — wow, it really does take all kinds — rather than a prompt for cosmic horror. And it gave me confidence that, if I

Bugs

Reliability when running things for a long period of time is paramount. Overall it's been a pretty smooth ride but I ran into this specific error during a handful of nights which stopped the process. I hope they get to the bottom of it and solve it as I'm not the only one to report it!

```
RangeError: Maximum call stack size exceeded

file:///usr/local/bin/claude_code/node_modules/@anthropic-ai/claude-code/cli.js:
3094
`)||J.includes("**")||J.includes("*"))return;if(X.includes("prompt is too long")
||X.includes("context length")||X.includes("token limit"))return;if(X.includes("
thanks")||X.includes("thank you")||X.includes("looks good")||X.includes("that wo
rked")||X.includes("that's all"))return;A.toolUseContext.setAppState((W)=>({...W
,promptSuggestion:{text:J,shownAt:Date.now()}}));let I=G.totalUsage.input_tokens
+G.totalUsage.cache_creation_input_tokens+G.totalUsage.cache_read_input_tokens;Q
A("tengu_prompt_suggestion_shown",{source:"forked_agent",...I>0&&{cacheHitRate:G
```

This setup is far from optimal but has worked so far. Hopefully this gets streamlined in the future!

Porting Pokemon

One Shot

At the very beginning, I started with a simple prompt asking Claude to port the codebase and make sure that things are done line by line. At first it felt extremely impressive, it generated thousands of lines of Rust that was compiling.

Sadly it was only an appearance as it took a lot of shortcuts. For example, it created two different structures for what a move is in two different files so that they would both compile independently but didn't work when integrated together. It ported all the functions very loosely where anything that was remotely complicated would not be ported but instead "simplified".

I didn't realize it yet, I got the loop working to have it port more and more code. The issue is that it created wrong abstractions all over the place and kept adding hardcoded code to make whatever it was supposed to fix work. This wasn't going to go anywhere.

Giving it structure

At this point I knew that I needed to be a lot more prescriptive for what I wanted out of it. Taking a step back, the end result should have every JavaScript file and every method inside to have a Rust equivalent.

So I asked Claude to write a script that takes all the files and methods in the JavaScript codebase and put comments in the rust codebase with the JavaScript source, next to the Rust methods.

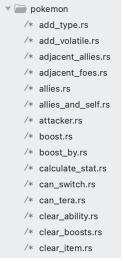
It was really important for it to be a script as even when instructed to copy code over, it would mistranslate JavaScript code. Being deterministic here greatly increased the odds of getting the right results.

```
/// onStart(pokemon, source, effect) {
///     if (effect && ['Electromorphosis', 'Wind Power'].includes(effect.name)) {
///         this.add('-start', pokemon, 'Charge', this.activeMove!.name, '[from] ability: ' + effect.name);
///     } else {
///         this.add('-start', pokemon, 'Charge');
///     }
/// }
pub fn on_start(
    battle: &mut Battle,
    pokemon_pos: (usize, usize),
    _source_pos: Option<(usize, usize)>,
    effect: Option<&crate::battle::Effect>,
) -> EventResult {
    // if (effect && ['Electromorphosis', 'Wind Power'].includes(effect.name)) {
    let is_special_ability = if let Some(eff) = effect {
        eff.name == "Electromorphosis" || eff.name == "Wind Power"
    } else {
        false
    };
```

Litte Islands

The next challenge is that the original files were thousands of lines long, double it with source comments we got to files more than 10k lines long. This causes a ton of issues with the context window where Claude straight up refuses to open the file. So it started reading the file in chunks but without a ton of precision. Also the context grew a lot quicker and compaction became way more frequent.

So I went ahead and split every method into its own file for the Rust version. This dramatically improved the results. For maximal efficiency I would need to do the same for the JavaScript codebase as well but I was too afraid to do it and accidentally change the behavior so decided not to.



Cleanup

The process of porting went through two repeating phases. I would give a large task to Claude to do in a loop that would churn on it for a day, and then I would need to spend time cleaning up the places where it went into the wrong direction.

3. If it were a good idea, someone would be doing it already.

There's a belief, especially common among wonky types, that you don't find $20 bills on the sidewalk — if an opportunity were real, someone would have seized it already. This sounds worldly and wise, but it's actually a form of defensive thinking. It assumes the current way is best, that everything worth doing is being done.

In my experience, this is just false. When I started playing poker, I discovered that physical tells were basically a wide-open field — despite the fact that most pros believed the topic was played out. If you think you see an opportunity others have missed, you should have a story for why they're missing it, and you should ask around to stress-test that story. But if no one can give you a satisfying reason, don't assume one exists. Sometimes the answer really is just inertia or fear.

4. If it's worth doing, it's worth doing well.

Almost nothing is worth doing "well." Many things can and should be done to a minimal standard of quality, because anything more is a waste of effort that can be better allocated. The exceptions aren't things that should be done "well," but things that should be done to an exceptionally high standard, probably a much higher one than you see modeled by the people around you.

This is not semantics; most people really "do everything the way they do anything" — whether that's in a low-effort or high-effort way. But the point of phoning it in on a lot of stuff is to save your energy to expend on the 1 in 1000 things that really matter … and then to actually spend it.

A recent example: I decided that I want to narrate my own audiobook for *You Can Just Do Things*. When I told a friend in the industry this and asked whether they knew any good coaches to work with, they said that hiring a coach wasn't normal or necessary, because my publisher would decide based on a sample whether I was good enough at reading to do it, and if so give me a producer to work with day-of. But it "1 in 1000" matters to me whether the audiobook is so good no one would ever think of returning it on account of the narration, so I'm not interested in the normal-shaped solution that constitutes doing it "well," I'm interested in the one that causes me to come to mind when someone asks this question. In this case, that means hiring multiple coaches and practicing for months.

1. What doesn’t kill you makes you stronger.

When I was 31, my husband of four years, who I loved more than my own life, left me completely out of the blue. We’d had an incredible, epic romance — drawn together like magnets, we married on our third date and spent the years that followed burrowing ever closer together. Maybe that was the problem — I can’t say for sure, because I never got a satisfying explanation out of him. One day I was the protagonist of a Princess Bride-worthy love story; 48 hours later, he moved out.

From the time we met, I had never contemplated the possibility of living without him, and I didn’t want to. I wanted to die for a long time — at least a couple of years. But as I slowly learned to sequester away thoughts of him and the emotions they brought up, I became convinced of a silver lining: The experience would make me impenetrable. If I survived it, I would know I could survive anything, and nothing would be able to hurt me again.

It took the better part of a decade and another deeply identity-shifting relationship, with the man who became my second husband, for me to see that I had it all wrong: Impenetrability means avoiding the parts of life that are most life-like. There’s a reason the visual metaphor for love is an arrow.

The near-fatal catastrophes of our lives don’t always make us stronger. Surviving one hard thing does provide evidence that you can survive other hard things, which is easy to *mistake* for getting stronger. But in reality they often just make us brittle, make us dissociate and flee from the situations that remind us we can be broken.

2. When you know, you know.

Mario Gabriele of the Generalist had a great “50 Things” style post a few months ago that I found myself nodding along to almost the whole way through. The glaring exception was Number 33: “Love is easy. The difference between bad relationships and meeting your spouse is not a matter of degree, but category. Compatibility is unignorable and effortless.”

I believed this completely, once. I’ve since come to believe that what’s unignorable and effortless is chemistry. Chemistry can be powerful enough to turn your life sideways, but it’s not the same thing as compatibility. Compatibility is litigated on the timescale of years and decades; it’s not a momentary feeling but a description of what happens during an extended course of change. Do you grow toward one another, or do you grow apart? Does the relationship help you move closer to the person you aspire to be?

I “just knew” with my first husband. I wasn’t sure with my second. Four years in, the signs have flipped.

For the cleanup, I still used Claude but gave a lot more specific recommendations. For example, I noticed that it would hardcode moves/abilities/items/... behaviors everywhere in the code when left unchecked, even after explicitly telling it not to. So I would manually look for all these and tell it to move them into the right places.

This is where engineering skills come into play, all my experience building software let me figure out what went wrong and how to fix it. The good part is that I didn't have to do the cleanup myself, Claude was able to do it just fine when directed to.

Integration

Build everything before testing

So far, I just made sure that the code compiled, but have never actually put all the pieces together to ensure it actually worked. What Claude really wanted was to do a traditional software building strategy where you make "simple" implementations of all of the pieces and then build them up as time goes.

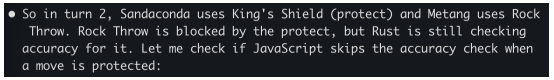
But in our case, all this iteration has already happened for 10 years on the pokemon-showdown codebase. It's counter productive to try and re-learn all these lessons and will unlikely converge the same way. What works better is to port everything at once, and then do the integration at the end once.

I've learned this strategy from working on Skip, a compiler. For years all the building blocks were built independently and then it all came together with nothing to show for but within a month at the end it all worked. I was so shocked.

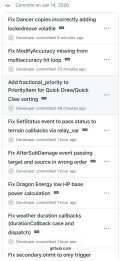
End-to-end test

Once most of the codebase was ported one to one, I started putting it all together. The good thing is that we can run and edit the code in JavaScript and in Rust, and the input/output is very simple and standardized: list of pokemons with their options (moves, items, nature, iv/ev spread...) and then the list of actions at each step (moves and switches). Given the same random sequence, it'll advance the state the same way.

Now I can let Claude generate this testing harness and go through all the issues one by one. Impressively, it was able to figure out all issues and fix them.



Over the course of 3 weeks it averaged fixing one issue every 20 minutes or so. It fixed hundreds of issues on its own. I never intervened, it was only a matter of time before it fixed every issue that it encountered.



Giving it structure

At the beginning, this process was extremely slow. Every time a compaction happened, Claude became "dumb" again and reinvented the wheel, writing down tons of markdown files and test scripts along the way. Or Claude decided to take the easy way out and just generate tons of tests but never actually making them match with JavaScript.

So, I started looking at what it did well and encoding it. For example, it added a lot of debugging around the PRNG steps and what actions happened at every turn with all the debugging metadata. So I asked it to create a single test script to print down this information for a single step and to print stack traces. Then add instruction to the Claude.md file. This way every investigation started right away.

The long slog

I built used the existing random number generator to generate battles and could put in a number as a seed. This let me generate consistent battles at an increasing size.

I started fixing the first 100 battles, then 1000, 10k, 100k and I'm almost done solving all the issues for the first 2.4 million battles! I'm not sure how many more issues there are but the good thing is that they are getting smaller and smaller as the batch size increases.

I've tried asking Claude to optimize it further, it created a plan that looks reasonable (I've never interacted with Rust in my life) and it spent a day building many of these optimizations but at the end of the day, none of them actually improved the runtime and some even made it way worse.

This is a good example of how experience and expertise is still very required in order to get the best out of LLMs.

Conclusion

This is pretty wild that I was able to port a ~100k lines codebase from JavaScript to Rust in two weeks on my own with Claude Code running 24 hours a day for a month creating 5k commits! I have never written any line of Rust before in my life.

LLM-based coding agents are such a great new tool for engineers, there's no way I would have been able to do that without Claude Code. That said, it still feels like a tool that requires my engineering expertise and constant babysitting to produce these results.

Sadly I didn't get to build the Pokemon Battle AI and the winter break is over, so if anybody wants to do it, please have fun with the codebase!

The lies I used to tell myself

CATE HALL · 23 JAN 2026 · SOURCE

Epilogue

It works

🤯

I didn't quite know what to expect coming into this project. They usually tend to die due to the sheer amount of work needed to get anywhere close to something complete. But not this time!

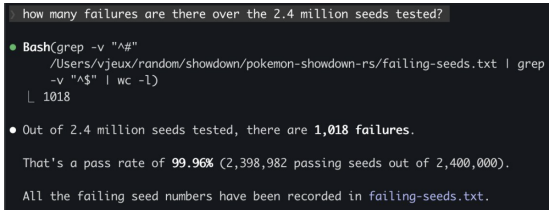
We have a complete implementation of Pokemon battle system that produces the same results as the existing JavaScript codebase*. This was done through 5000 commits in 4 weeks and the Rust codebase is around 100k lines of code.

*I wish we had 0 divergences but right now there are 80 out of the first 2.4 million seeds or 0.003%. I need to run it for longer to solve these.

Is it fast?

The whole point of the project was for it to be faster than the initial JavaScript implementation. Only towards the end of the project where we had a sizable amount of battles running perfectly I felt like it would be a fair time to do a performance comparison.

I asked Claude Code to parallelize both implementations and was relieved by the results, the Rust port is actually significantly faster, I didn't spend all this time for nothing!



Types of issues

There are two broad classes of issues that were fixed. The first one that I expected is that Rust has different constraints than JavaScript which need to be taken into account and lead to bugs:

- Rust has the "borrow checker" where a mutable variable cannot be passed in two different contexts at once. The problem is that "Pokemon" and "Battle" have references to each others. So there's a lot of workarounds like doing copies, passing indices instead of the object, providing functions with mutable object as callback...

- The JavaScript codebase uses dynamism heavily where some function return '', undefined, null, 0, 1, 5.2, Pokemon... which all are handled with different behaviors. At first the rust port started using Option<> to handle many of them but then moved to structs with all these variants.

- Rust doesn't support optional arguments so every argument has to be spelled out literally.

But the second one are due to itself... Claude Code is like a smart student that is trying to find every opportunity to avoid doing the hard work and take the easy way out if it thinks it can get away with it.

- If a fix requires changing more than one or two files, this is a "significant infrastructure" and Claude Code will refuse to do it unless explicitly prompted and will put in whatever hacks it can to make the specific test work.

- Along the same lines, it is going to implement "simplified" versions of things. For some methods, it was better to delete everything and asking it to port it over from scratch than trying to fix all the existing code it created.

- The JavaScript comments are supposed to be the source of truth. But Claude is not above changing the original code if it feels like this is the way to solve the problem...

- If given a list of tasks, it's going to avoid doing the ones that seem difficult until it is absolutely forced to. This is inefficient if not careful as it's going to keep spending time investigating and then skipping all the "hard" ones. Compaction is basically wiping all its memory.

Prompts

I didn't write a single line of code myself in this project. I alternated between "co-op" where I work with Claude interactively during the day and creating a job for it to run overnight. I'll focus on the night ones for this section.

Conversion

For the first phase of the project, I mostly used variations of this one. Asking it to go through all the big files one by one and implement them faithfully (it didn't really follow instructions as we've seen later...)

> Open BATTLE_TODO.md to get the list of all the methods in both battle*.rs.
> Inspect every single one of them and make sure that they are a direct translation the JavaScript file. If there's a method with the same name, the JavaScript definition will be in the comment.
> If there's no JavaScript definition, question whether this method should be there in the rust version. Our goal is to follow as closely as possible the JavaScript version to avoid any bugs in translation. If you notice that the implementation doesn't match, do all the refactoring needed to match 1 to 1.
> This will be a complex project. You need to go through all the methods one by one, IN ORDER. YOU CANNOT skip a method because it is too hard or would requiring building new infrastructure. We will call this in a loop so spend as much time as you need building the proper infrastructure to make it 1 to 1 with the JavaScript equivalent. Do not give up.
> Update BATTLE_TODO.md and do a git commit after each unit of work.

Todos

Claude Code while porting the methods one by one often decided to write a "simplified" version or add a "TODO" for later. I also found it to be useful when generating work to add the instructions in the codebase itself via a TODO comment, so I don't need to wish that it's going to be read from the context.

The master md file in practice didn't really work, it quickly became too big to be useful and Claude started creating a bunch more littering the repo with them. Instead I gave it a deterministic way to go through then by calling grep on the codebase, so it knew when to find them.

> We want to fix every TODO in the codebase. `TODO` or `simplif` in pokemon-showdown-rs/.
> There are hundreds of them, so go diligently one by one. Do not skip them even if they are difficult. I will call this prompt again and again so you don't need to worry about taking too long on any single one.
> The port must be exactly one to one. If the infrastructure doesn't exist, please implement it. Do not invent anything.
> Make sure it still compiles after each addition and commit and push to git.

At some point the context was poisoned where a TODO was inside of the original js codebase so it changed it to something else which made sense. But then it did the same for all the subsequent TODOs which didn't... Thankfully I could just revert all these commits.

Fixing

I put in all the instructions to debug in Claude.md and a script to run all the tests which outputs a txt file with progress report. This way Claude was able to just keep going fixing issues after issues.

> We want to fix all the divergences in battles. Please look at 500-seeds-results.txt and fix them one by one. The only way you can fix is by making sure that the differences between javascript and rust are explained by language differences and not logic. Every line between the two must match one by one. If you fixed something specific, it's probably a larger issue, spend the time to figure out if other similar things are broken and do the work to do the larger infrastructure fixes. Make sure it still compiles after each addition and commit and push to git. Check if there are other parts of the codebase that make this mistake.

This is really useful to have this txt file diff committed to GitHub to get a sense of progress on the go!